

Introducción Teórica

En el desarrollo de aplicaciones con MongoDB, existen dos enfoques principales para gestionar la comunicación:

1. **Driver Nativo:** Provee la comunicación más directa con el motor de base de datos. Se destaca por ofrecer el máximo control sobre las consultas y una mayor eficiencia al no tener capas intermedias.
2. **Mongoose (ODM):** Actúa como una capa de abstracción sobre el driver. Permite definir esquemas para organizar los datos y forzar validaciones directamente en el código de la aplicación.

Parte 1: Implementación con Driver Nativo

El driver oficial (`mongodb`) es ideal para aplicaciones sencillas o procesos de alto volumen donde el rendimiento es crítico.

Instrucciones:

1. Inicialicen el proyecto e instalen el paquete: `npm install mongodb`.
2. Creen un archivo `driver_lab.js` para conectarse a la base de datos `laboratorio`.
3. Usando el método `insertMany`, carguen un array con tres sensores de robótica (ej: Sensor de Humedad, Servo Motor, LED RGB).

Parte 2: Implementación con Mongoose (ODM)

Mongoose ayuda a mantener el orden y la coherencia en aplicaciones de mediana o gran escala.

Instrucciones:

1. Instalen el paquete: `npm install mongoose`.
2. Definan un **Esquema** (`Schema`) para los componentes que incluya:
 - `nombre`: Tipo `String` y obligatorio (`required: true`).
 - `stock`: Tipo `Number`.
3. Activen la opción `timestamps: true` para generar automáticamente los campos de fecha de creación y actualización.
4. Creen el modelo y realicen una inserción de prueba.

Parte 3: Reflexión Final

Basándose en lo aprendido, respondan:

1. ¿Cuál es la función del driver como "traductor" entre la aplicación y MongoDB?.
2. ¿A qué formato convierte el driver las estructuras de datos nativas (como objetos de JavaScript) para enviarlas al servidor?.
3. Según lo visto, ¿en qué situación de un proyecto de ingeniería elegirían usar el driver nativo en lugar de Mongoose?.